

Software- modernisierungs- strategien

*32 Ansätze, um Legacy-Systeme
zu revitalisieren und gezielt
an neue Business-Anforderungen
anzupassen*

Kategorien:



Geschäftlich



Entwicklung



Infrastruktur



Befähigung



Begleitseite:

**feststelltaste.github.io/
littlestmodernization**



Ignorieren!

Fass dein System nicht an, solange es seinen Job gut erledigt.

Nicht jedes Legacy-System braucht Aufmerksamkeit. Manche laufen zuverlässig, verursachen keine Reibung und kosten wenig im Betrieb. Modernisierung um ihrer selbst willen verschwendet Ressourcen und bringt Risiko in etwas, das gut funktioniert. Die entscheidende Frage ist nicht "Können wir das verbessern?" sondern "Sollten wir es überhaupt?"



Beatmen!

Halte dein Legacy System am Leben, aber entscheide es bewusst und nicht aus Gewohnheit.

Beatmen liegt zwischen Ignorieren und Abschalten. Das System hat seinen Zenit überschritten und alle wissen es, aber Ersetzen oder Stilllegen ist noch nicht machbar. Die Strategie akzeptiert die laufenden Betriebskosten als das kleinere Übel und konzentriert sich darauf, das System stabil genug zu halten, um seinen verbleibenden Zweck zu erfüllen, ohne in Verbesserungen zu investieren, die es nie brauchen wird.



Abwälzen!

Gib an andere ab, was dich aufhält, und konzentriere dich auf das, was wirklich zählt.

Abwälzen beschreibt die Übertragung von Verantwortung für Teile eines Softwaresystems an eine externe Partei, etwa einen Anbieter, einen Managed-Service-Provider oder eine Plattform. Der Ansatz eignet sich, wenn interne Teams nicht die Kapazität, das Fachwissen oder das strategische Interesse haben, bestimmte Aufgaben selbst zu übernehmen. Ziel ist es, interne Ressourcen freizusetzen, indem jemand anderes betreibt und verantwortet, was nicht zum Kerngeschäft gehört.



Verschenken!

Veröffentliche, was du nicht mehr alleine voranbringen willst, und lass die Community die Last mittragen.

Verschenken beschreibt die Entscheidung, ein Softwaresystem oder eine Komponente als Open Source zu veröffentlichen, damit eine externe Community die Weiterentwicklung und Pflege mitträgt. Sinnvoll ist das, wenn die Organisation die Kosten für die Weiterentwicklung alleine nicht mehr rechtfertigen kann, das System aber für andere noch einen Wert hat. Ziel ist es, die Last der Alleinpflege zu reduzieren und gleichzeitig das System am Leben zu erhalten.



Verkaufen!

Verkauf dein Legacy System an jemanden,
für den es mehr wert ist als für dich.

Verkaufen beschreibt die Übertragung der Eigentumsrechte an einem Legacy-System an eine andere Organisation, für die es einen höheren strategischen oder wirtschaftlichen Wert hat. Sinnvoll ist das, wenn das System nicht mehr zur Ausrichtung der Organisation passt, aber noch ein funktionierendes Produkt mit bestehender Nutzerbasis darstellt. Ziel ist es, den Wert des Systems zu realisieren und gleichzeitig die laufenden Kosten und Verantwortung für die Pflege loszuwerden.



Shoppen!

Kauf eine fertige Lösung, wenn jemand anderes dein Problem bereits gelöst hat.

Shoppen beschreibt die Entscheidung, ein Legacy-System durch eine kommerziell verfügbare Standardsoftware zu ersetzen, anstatt es selbst zu bauen oder weiterzupflegen. Der Ansatz eignet sich, wenn eine Anbieterlösung die benötigte Funktionalität bereits gut abdeckt und die Anforderungen der Organisation sich nicht wesentlich vom allgemeinen Markt unterscheiden. Ziel ist es, den Aufwand für kundenspezifische Eigenentwicklung in Bereichen zu eliminieren, die das Unternehmen nicht differenzieren.



Abschalten!

Beende den Betrieb deines Legacy Systems geplant, geordnet und ohne Drama.

Abschalten beschreibt die geplante Stilllegung eines Legacy-Systems, das nicht mehr benötigt wird oder tragfähig ist. Der Ansatz eignet sich, wenn die Nutzer, Anwendungsfälle oder der Geschäftskontext des Systems weggefallen sind oder der Weiterbetrieb mehr kostet als der verbleibende Nutzen rechtfertigt. Ziel ist es, den Betrieb des Systems geordnet zu beenden, sodass Daten gesichert oder migriert und Abhängigkeiten sauber aufgelöst werden.



Zusammenlegen!

Ersetze deinen Zoo an Systemen durch weniger, besser handhabbare Einheiten.

Zusammenlegen beschreibt die Reduzierung redundanter Systeme, indem mehrere überlappende Lösungen zu einer kleineren, besser handhabbaren Menge zusammengeführt werden.

Das kommt infrage, wenn eine Organisation mehrere Systeme angesammelt hat, die denselben Zweck über verschiedene Teams oder Abteilungen hinweg erfüllen. Ziel ist es, die operative Komplexität, den Wartungsaufwand und die Kosten für parallele Systeme zu reduzieren.



Standardisieren!

Lege fest, wie deine Systeme gebaut, verbunden und betrieben werden, und beende den Wildwuchs.

Standardisieren beschreibt die Definition und Durchsetzung gemeinsamer Muster, Konventionen und Schnittstellen über eine Systemlandschaft hinweg. Sinnvoll ist das, wenn verschiedene Systeme auf inkompatible Weisen entwickelt wurden, was Integration, Einarbeitung und Wartung unnötig schwierig macht. Ziel ist es, die Vielfalt in der Art, wie Systeme gebaut und betrieben werden, zu reduzieren, damit Teams zwischen ihnen wechseln können, ohne alles neu lernen zu müssen.



Harmonisieren!

Stimme Systeme, Modelle und Prozesse aufeinander ab, damit sie reibungslos zusammenarbeiten.

Harmonisierung bedeutet, Systeme, die unabhängig voneinander entstanden sind, zur Zusammenarbeit zu bringen, ohne sie zu ersetzen. Statt Uniformität zu erzwingen, schafft man Abstimmung an den Grenzen: gemeinsames Vokabular, kompatiblen Datenaustausch und abgestimmte Ergebnisse. Die Systeme bleiben verschieden, aber sie widersprechen sich nicht mehr.



Kooperieren!

Arbeite mit externen Partnern zusammen,
um dein System gemeinsam
weiterzuentwickeln.

Kooperieren beschreibt, ein Softwaresystem gemeinsam mit einer externen Organisation weiterzuentwickeln, anstatt alles intern zu erledigen oder an einen Dienstleister abzugeben. Der Ansatz eignet sich, wenn ein System über das hinausgeht, was eine Organisation alleine voranbringen kann, aber Wissen, Risiko oder Investition es wert sind, sie mit einem Partner zu teilen. Ziel ist es, Kompetenzen zu bündeln und den Modernisierungsaufwand auf mehrere Schultern zu verteilen.



Zurückholen!

Hole die Kontrolle über dein System zurück, wenn Outsourcing mehr kostet als es bringt.

Zurückholen beschreibt die Entscheidung, eine Outsourcing-Vereinbarung zu beenden und die Softwareentwicklung wieder ins eigene Haus zu holen. Das kommt infrage, wenn die Kosten, Inflexibilität oder der Wissensverlust durch externe Dienstleister die ursprünglichen Vorteile der Auslagerung überwiegen. Ziel ist es, die direkte Kontrolle über ein System zurückzugewinnen, das für die Organisation strategisch bedeutsam ist.



Überdenken!

Stelle nicht dein System in Frage, sondern die Annahmen, auf denen es gebaut wurde.

Überdenken beschreibt die grundlegende Infragestellung der Annahmen, auf denen ein Legacy-System aufgebaut wurde, anstatt am System selbst iterativ weiterzuarbeiten. Sinnvoll ist das, wenn sich das ursprüngliche Problem, für das das System entworfen wurde, so grundlegend verändert hat, dass schrittweise Verbesserung bedeuten würde, das Falsche zu optimieren. Ziel ist es herauszufinden, ob überhaupt das richtige System gebaut wird, bevor weitere Investitionen in die Modernisierung fließen.



Umgestalten!

Richte die Struktur deines Systems an der Fachlichkeit aus, wie du sie heute verstehst.

Softwarestrukturen kodieren die Annahmen derer, die sie gebaut haben. Mit der Zeit werden diese Annahmen überholt: das Business entwickelt sich, Teams reorganisieren sich, das Domänenwissen vertieft sich. Umgestalten schließt die Lücke zwischen der Struktur des Systems und der heutigen fachlichen Realität, damit der Code leichter zu lesen, zu ändern und zu erweitern ist, ohne dass sich ändert, was das System tut.



Entrümpeln!

Entferne, was niemand mehr braucht, und gib deinem System seinen eigentlichen Zweck zurück.

Entrümpeln beschreibt die gezielte Entfernung von Features, Abhängigkeiten und Code aus einem Legacy-System, die nicht mehr benötigt werden. Das kommt infrage, wenn ein System über seinen eigentlichen Zweck hinausgewachsen ist und das angesammelte Gewicht ungenutzter oder redundanter Teile es schwerer macht, das System zu verstehen und zu ändern. Ziel ist es, den Fokus wiederherzustellen, indem alles gestrichen wird, was nicht aktiv zur Kernfunktion des Systems beiträgt.



Verbessern!

Halte dein System durch kontinuierliche, unspektakuläre Arbeit fit, bevor die Schulden dich einholen.

Verbessern beschreibt die laufende, schrittweise Investition in ein Legacy-System, um es gesund zu halten, ohne eine groß angelegte Transformation anzugehen. Typisch ist der Einsatz, wenn das System grundsätzlich solide ist, aber technische Schulden, veraltete Abhängigkeiten oder schlechtere Codequalität angesammelt hat. Ziel ist es, weitere Verschlechterung zu verhindern, indem die interne Qualität durch kleine, stetige Verbesserungen kontinuierlich gesteigert wird.



Übersetzen!

Behalte die bewährte Logik deines Systems und tausche nur die Technologie aus, in der sie steckt.

Übersetzen beschreibt die Konvertierung eines Legacy-Systems von einer Technologie in eine andere, wobei das bestehende Verhalten und die Geschäftslogik erhalten bleiben. Relevant wird das, wenn die zugrundeliegende Logik eines Systems noch solide ist, die Sprache, das Framework oder die Laufzeitumgebung aber zur Belastung geworden sind. Ziel ist es, das System auf einen wartbaren Technologie-Stack zu bringen, ohne die über die Zeit angesammelte Logik neu schreiben zu müssen.



Abspalten!

Trenne einen Teil deines Systems ab und baue daraus eine eigenständige, neue Lösung.

Abspalten beschreibt die Erstellung einer unabhängigen Kopie einer Codebasis, die dann getrennt vom Original weiterentwickelt wird. Das kommt infrage, wenn ein Teil des Systems in eine grundlegend andere Richtung entwickelt werden muss, etwa um einen neuen Markt oder Anwendungsfall zu bedienen, ohne das Original einzuschränken oder zu stören.

Ziel ist es, einem eigenständigen Produkt oder Anwendungsfall seinen eigenen Entwicklungspfad zu geben.



Umwidmen!

Setz dein System für eine neue Zielgruppe ein, statt es für einen schrumpfenden Markt weiterzupflegen.

Umwidmen beschreibt den Einsatz eines bestehenden Systems für eine neue Zielgruppe, einen neuen Markt oder einen neuen Anwendungsfall, für den es ursprünglich nicht gedacht war. Typisch ist der Einsatz, wenn die Kernfähigkeiten des Systems gut zu einem anderen Kontext passen und die Kosten der Anpassung geringer sind als ein Neubau. Ziel ist es, die Nutzungsdauer des Systems zu verlängern, indem ein neues Problem gefunden wird, das es lösen kann.



Plündern!

Behalte die wertvollen Komponenten
deines Legacy Systems und lass den Rest
hinter dir.

*Legacy-Systeme sind selten gleichmäßig
schlecht. Unter den veralteten Oberflächen
und verworrenen Abhängigkeiten stecken oft
Komponenten, die Jahre harter Verfeinerung
verkörpern: kampferprobte Geschäftsregeln,
Algorithmen, die gegen reale Grenzfälle
getunt wurden, Datenmodelle, die unzählige
Anforderungsänderungen überlebt haben.
Plündern bedeutet, selektiv vorzugehen:
mitnehmen, was seinen Platz verdient hat,
den Rest zurücklassen.*



Umhüllen!

Lass den Kern unangetastet und modernisiere nur, wie dein System mit der Außenwelt redet.

Umhüllen beschreibt das Hinzufügen moderner Schnittstellen und Integrationsschichten um ein Legacy-System, ohne seine interne Logik zu verändern. Relevant wird das, wenn der Kern eines Systems noch funktioniert, die Art und Weise, wie es mit der Außenwelt kommuniziert, aber veraltet, inkompatibel oder schwer handhabbar geworden ist. Ziel ist es, das System für moderne Konsumenten und Werkzeuge zugänglich zu machen, ohne den darunter laufenden Code anzufassen.



Neubauen!

Bau neu, wenn das alte System mehr Last ist als Wert.

Neubauen beschreibt den vollständigen Neubau eines Legacy-Systems von Grund auf, wobei die bestehende Codebasis verworfen wird. Relevant wird das, wenn das System so weit verfallen ist, dass schrittweise Verbesserung nicht mehr tragfähig ist, oder wenn Technologie und Architektur so grundlegend mit den Anforderungen unvereinbar sind, dass kein anderer Ansatz greift. Ziel ist ein neues System, das heutige Anforderungen erfüllt, geleitet von allem, was aus dem alten gelernt wurde.



Zerlegen!

Schneide dein System entlang seiner natürlichen Grenzen, damit eine Änderung nicht mehr das ganze System gefährdet.

Zerlegen beschreibt die Aufteilung eines großen, eng gekoppelten Systems in kleinere, unabhängig deploybare Einheiten. Typisch ist der Einsatz, wenn ein monolithisches System so gewachsen ist, dass Änderungen in einem Bereich inakzeptable Risiken oder Verzögerungen in anderen verursachen. Ziel ist es, klare Grenzen zwischen Systemteilen zu etablieren, sodass jeder Teil nach eigenen Bedingungen entwickelt, deployt und skaliert werden kann.



Rausholen!

Hole das Wissen aus deinem System heraus, bevor du den nächsten Schritt machst.

Rausholen beschreibt den Prozess, das in einem Legacy-System eingebettete Wissen zu bergen und zu dokumentieren, bevor weitere Modernisierungsschritte unternommen werden.

Relevant wird das, wenn das System Geschäftsregeln, Domänenlogik oder operatives Wissen enthält, das nirgendwo sonst existiert und verloren ginge, wenn das System ohne Sorgfalt ersetzt oder neu geschrieben würde. Ziel ist es, implizites Wissen explizit zu machen, damit es den nächsten Schritt informieren kann.



Präservieren!

Konserviere dein System, damit du es bei Bedarf noch starten kannst, ohne es aktiv zu pflegen.

Manche Systeme überleben ihre operative Nutzungsdauer, können aber nicht einfach entsorgt werden. Rechtliche Verpflichtungen, Prüfpfade oder historische Aufzeichnungen können es erfordern, dass sie über Jahre hinweg zugänglich bleiben. Präservierung friert das System in einem startbaren Zustand ein, typischerweise durch Containerisierung, Emulation oder dokumentierte Umgebungs-Snapshots, damit es bei Bedarf gestartet werden kann, ohne dass es jemand aktiv pflegt.



Umstellen!

Stell dein System auf ein neues Fundament, wenn der Code noch stimmt aber das darunter nicht mehr.

Umstellen beschreibt die Migration eines Systems auf ein neues technisches Fundament, etwa eine andere Laufzeitumgebung, Datenbank oder Cloud-Plattform, während die Anwendungslogik weitgehend unverändert bleibt. Typisch ist der Einsatz, wenn die Infrastrukturschicht zum Engpass oder Risiko geworden ist, das System selbst aber nicht neu geschrieben werden muss. Ziel ist es, dem System eine tragfähigere Basis zu geben, ohne die Kosten und Risiken einer vollständigen Ablösung.



Umziehen!

Bring dein System in eine neue Umgebung und lass alles andere wie es ist.

Umziehen, oft auch "Lift and Shift" genannt, verschiebt ein System in eine neue Infrastrukturmgebung, ohne den Anwendungscode anzufassen. Es ist der schnellste verfügbare Modernisierungsschritt: kein Refactoring, keine Neuarchitektur, kein Umlernen. Der Kompromiss: Alle Ineffizienzen und Einschränkungen, die in der Anwendung selbst stecken, ziehen mit in die neue Umgebung.



Werkzeuge modernisieren!

Bring dein Tooling auf den Stand der Zeit, damit Legacy-Arbeit effizienter wird.

Werkzeuge modernisieren beschreibt die Modernisierung der Entwicklungswerkzeuge, Build-Systeme, Testinfrastruktur und operativen Tooling, die für die Arbeit mit einem Legacy-System verwendet werden. Der Ansatz bietet sich an, wenn das Tooling rund um ein System so veraltet ist, dass es unnötige Reibung erzeugt, Feedback-Schleifen verlangsamt oder Risiken im täglichen Betrieb einführt. Ziel ist es, die Arbeit mit dem System weniger schmerzhaft zu machen, ohne das System selbst zu verändern.



Kurs korrigieren!

Richte die strategische Ausrichtung
deines Unternehmens auf einen
realistischen Umgang mit Legacy aus.

Kurs korrigieren beschreibt, eine klare, gemeinsame Ausrichtung für den Umgang einer Organisation mit Legacy-Modernisierung zu etablieren. Der Ansatz bietet sich an, wenn Modernisierungsvorhaben durch Trend, Bauchgefühl oder lokale Initiative vorangetrieben werden statt durch ein kohärentes Verständnis dessen, was die Organisation tatsächlich braucht. Ziel ist, dass Modernisierungsinvestitionen bewusst und im Verhältnis zu den echten strategischen Prioritäten des Unternehmens getätigt werden.



Neu aufstellen!

Forme Teams, Verantwortlichkeiten und Anreizsysteme so um, dass Legacy-Arbeit endlich gut funktioniert.

Technologie allein modernisiert keine Systeme, Menschen tun es. Und Menschen arbeiten in Organisationsstrukturen, die gute Arbeit entweder ermöglichen oder blockieren. Wenn Teamgrenzen quer zu natürlichen Systemgrenzen verlaufen, wenn niemand die schwierigen Teile verantwortet oder wenn Anreize neue Features über Wartung stellen, stockt die Modernisierung unabhängig von der gewählten technischen Strategie. Neu aufstellen bedeutet, die strukturellen Ursachen dieser Reibung zu beseitigen.



Schlau machen!

Investiere in die Menschen, die du schon hast, bevor du in neue Technologie investierst.

Schlau machen beschreibt die gezielte Investition in das Wissen und die Fähigkeiten der Menschen, die mit einem Legacy-System arbeiten. Der Ansatz bietet sich an, wenn dem Team die Kompetenzen fehlen, um das System effektiv zu verstehen, zu pflegen oder zu modernisieren, und diese Lücke ein größeres Hindernis ist als der Zustand der Technologie selbst. Ziel ist es, die Fähigkeit des Teams im Umgang mit dem System zu steigern, bevor weitere Mittel in Werkzeuge oder Neubauten fließen.



Prozesse entrostet!

Verbessere nicht das System, sondern die Art, wie dein Team damit arbeitet.

Prozesse entrostet beschreibt die Modernisierung der Arbeitsabläufe und Kollaborationsmuster, die ein Team beim Arbeiten mit einem Legacy-System verwendet, anstatt das System selbst zu verändern. Der Ansatz bietet sich an, wenn langsame Lieferzeiten, schlechte Qualität oder hohe Reibung durch die Arbeitsweise des Teams verursacht werden und nicht durch den Code. Ziel ist es, prozessbedingte Hindernisse zu beseitigen, die das Team daran hindern, effektiv mit dem System zu arbeiten.